Software Requirements Specification

for

Voting System

Version 1.0 approved

Prepared by
Ly Xiong
Cindy Su
Elsdon Chang
Kongmeng Thao

Team 2

Feb 17, 2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 1
1.4 Product Scope 1
1.5 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Classes and Characteristics 2
2.4 Operating Environment 2
2.5 Design and Implementation Constraints 2
2.6 User Documentation 2
2.7 Assumptions and Dependencies 3
3. External Interface Requirements 3
3.1 User Interfaces 3
3.2 Hardware Interfaces 3
3.3 Software Interfaces 3
3.4 Communications Interfaces 3
4. System Features 4
4.1 System Feature 1 4
4.2 System Feature 2 (and so on) 4
5. Other Nonfunctional Requirements 4
5.1 Performance Requirements 4
5.2 Safety Requirements 5
5.3 Security Requirements 5
5.4 Software Quality Attributes 5
5.5 Business Rules 5
6. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Analysis Models 5
Appendix C: To Be Determined List 6

Revision History

Name	Date	Reason For Changes	Version
Cindy Su	2/19	Initial Draft for Section 3 and Part of Section 5	0.0
Kongmeng Thao	2/19	Initial Draft for Section 1	0.2

Elsdon Chang	2/19	Initial Draft for Section 2	0.4
Ly Xiong	2/19	Add and modify Section 4 for use cases	0.6
Ly Xiong	2/20	Modify Section 4 use-cases 4.5 to 4.8	0.8
Kongmeng Thao & Ly Xiong & Elsdon Chang & Cindy Su	2/20	Revise and edit the document preparation for the final finalization of Sections 1-5	0.9
Kongmeng Thao & Ly Xiong & Elsdon Chang & Cindy Su	2/21	Finalize the document and spell-check everything	1.0

1. Introduction

1.1 Purpose

This **software requirement specification (SRS)** document presents a detailed description and functionality of a new standalone voting system that handles **plurality voting** and **single transferable voting (STV)** for elections. This document explains the software's purpose, features, functionality, interfaces, and restraints.

1.2 Document Conventions

This document was created based on the IEEE template for system requirement specification Documents. Bold text is used for section headings, references, and key terms.

1.3 Intended Audience and Reading Suggestions

This document is intended for software developers, testers, and election officials. Readers new to the system should start with **Overall Description** and **System Features** to understand the software's objectives. Developers and testers should read **Other Nonfunctional Requirements, Other Requirements,** and **External Interface Requirements** for more detailed technical information about the system. Follow these approaches to quickly locate the most relevant information.

1.4 Product Scope

The voting system processes election ballots and determines winners based on either the plurality or STV algorithm. The system's goal is to make processing election results more efficient, accurate, and user-friendly for election officials. It can support up to 100,000 ballots, ensuring fair vote counting, and is consistent with the objective of providing a secure and reliable voting system.

1.5 References

IEEE Template for System Requirement Specification Documents:
https://canvas.umn.edu/courses/483118/files/50092534?module_item_id=13985225

Project1_Description_Waterfall_2025. Voting System Requirements and Specifications. [Project documentation]:
https://canvas.umn.edu/courses/483118/files/50177539?module_item_id=13993087

2. Overall Description

2.1 Product Perspective

The voting system is a new, self-contained product that handles the following methods of voting.

- Plurality voting:
  A voting method where the candidate with the most votes wins.

- STV:
  A voting system where voters rank candidates in order of preference. To win, the candidate must reach the **Droop Quota**.
  $$Droop\ Quota = \lfloor \frac{numberOfBallots}{numberOfSeats + 1} \rfloor + 1$$
  This style of voting ensures proportional representation and minimizes wasted votes.

2.2 Product Functions

- Input Handling: Prompts users to input election information in the terminal
- File Reading: Reads the inputted file and stores the data in a data structure
- Plurality Voting Algorithm: The algorithm for a Plurality election.
- STV with Droop Quota Algorithm: The algorithm for an STV election.
- Display Election Results: Prints election results to the terminal.
- Audit File Naming: Prompts user to input audit file name if STV method is selected.
- Audit File Generation: Includes election information
- Toggle Shuffle: Randomizes the order of ballots.

2.3 User Classes and Characteristics

- Election Officials: Users who will run the system to process election results.
- Testers: Users who will test the system with test files and toggle the shuffle feature to ensure unbiased results.

2.4 Operating Environment

- Ubuntu

2.5 Design and Implementation Constraints

The voting system is developed in Java. The system must process up to 100,000 ballots in under 5 minutes. The interface should be easy to use and learn, with training taking less than 15 minutes.

2.6 User Documentation

TBD

2.7 Assumptions and Dependencies

The system requires the user's system to install Java 21.0.6 or higher. Ballot files are assumed to be in CSV format and free of errors.

3. External Interface Requirements

3.1 User Interfaces

- The system will prompt the user for the voting method in the terminal.
- The system will prompt the user for the number of seats in the terminal.
- The system will prompt the user for the audit file name in the terminal if STV is selected.
- The system will prompt the user for a CSV file
- The system will display an error message to the user in the terminal and terminate the program if an invalid input is entered.
- After the system processes the votes successfully, a snapshot of the election results will be printed out in the terminal. The snapshot includes winners, losers, percentage of votes received if plurality, or order of the winning and losing for STV.

3.2 Hardware Interfaces

The system will run on CSE lab machines and will not require any specialized hardware interfaces.

3.3 Software Interfaces

The voting system is coded in Java version 21.0.6 and above, running on CSE lab machines with Ubuntu (Linux) as the operating system. The system also takes in a CSV file as input that contains ballot data and outputs an audit file containing a report of the election.

3.4 Communications Interfaces

The system does not require any external communication interfaces. It operates independently and processes local files.

4. System Features

This section outlines the use cases for the voting system, which supports both plurality voting and STV. The system allows election officials to input election details, process ballots, display results,

and generate an audit report. The following use cases describe the key functions and interactions with the system to make sure that the election results are legitimate and user experience.

4.1 Select Voting Algorithm

Name	Select Voting Algorithm
ID	UC_000
Description	The user selects which algorithm they want to use, Plurality or STV
Actors	Election officials, Testers
Organization Benefits	Allows the user to select the desired algorithm that best fits their needs
Frequency of Use	Once whenever the user wants to use a voting system
Triggers	The user starts the program
Preconditions	The system is running and ready to accept user input
Postconditions	The system is configured to use the selected voting algorithm
Main Course	1. The user runs the system 2. The system prompts the user to select a voting algorithm 3. The user selects the desired algorithm 4. The system proceeds to the next step (UC_001)
Alternate Courses	N/A
Exceptions	EX1: System Fail 1. The system displays an error message to the user to try again 2. rerun the system

4.2 Initial Input

Name	Initial Input
ID	UC_001
Description	The user inputs all the necessary information for the chosen algorithm,

	including the number of seats to be filled, a file that contains the ballots, as well as an audit file name and also on/off shuffle if STV is used
Actors	Election officials, Testers
Organization Benefits	Allows the user to input information based on the selected algorithm
Frequency of Use	Every time before processing selective the algorithm
Triggers	After an algorithm is selected (UC_000)
Preconditions	N/A
Postconditions	Save all the input information and ready to be processed
Main Course	1. Prompts the user to input all the necessary information as mentioned in the description 2. The system will read the provided information including the CSV file (UC_002)
Alternate Courses	AC1: Fail to read in the provided input 1. The system will display an error message to the user 2. Restart the program refer to UC_000
Exceptions	EX1: Invalid Input 1. The system will display an error message to the user 2. Return to main course step 1 EX2: System Fail 3. The system displays an error message to the user to try again 4. rerun the system UC_000

4.3 Read CSV File

Name	Read CSV file
ID	UC_002
Description	After inputting the CSV file containing ballots, the file is read to be prepared for processing
Actors	Tester or Election Official

Organization Benefits	Has all voter's ballots in one file, making it easier to determine the winner
Frequency of Use	Every time the program runs
Triggers	A valid file name was entered
Preconditions	The file name is inputted correctly The file is preprocessed in a valid format (STV or Plurality)
Postconditions	Data in the file is stored in a data structure and is ready to be processed
Main Course	1. System reads the file header to determine candidates 2. System stores this information for voting calculation
Alternate Courses	AC1: File missing information 1. The system displays an error message to the user to input a new file 2. Go back to UC_001
Exceptions	EX1: File is not in a valid format 1. The system notifies the user that an error occurred 2. Go back to UC_001 EX2: System Fail 5. The system displays an error message to the user to try again 6. rerun the system UC_000

4.4 Process STV Voting

Name	Process STV voting
ID	UC_003
Description	The STV voting algorithm is calculated using the Droop quota. Votes are redistributed based on quota calculations until all seats are filled.
Actors	Tester or Election Official
Organization Benefits	Allow ranked voting and ensure fairness by shuffling the ballots Reduces vote waste by redistributing them to the next candidates
Frequency of Use	Once every time the STV voting format is used
Triggers	The STV voting system is selected and necessary inputs are provided

Preconditions	A valid ballot file exists in the CSV file and the number of seats is provided
Postconditions	Display the election result, including the type of election, number of ballots, number of seats, number of candidates, as well as the winners and losers along with their percentage of votes received. An audit file is created as a text file and saved
Main Course	1. The system reads the ballots file (see UC_002) and other inputs 2. The system shuffles the ballots as default (unless shuffle is turned off for testing, see UC_006) 3. The system calculates the Droop quota 4. The system distributes ballots to candidates based on first-choose preferences 5. Candidates who reach the quota are declared winners, and the rest of their votes are redistributed 6. If no candidate reaches the quota, then the candidate with the lowest votes is eliminated, and their votes are redistributed 7. Repeat steps 5 to 6 until all seats are filled 8. The system displays the result 9. The system creates and saves the audit file (see UC_005)
Alternate Courses	AC1: Shuffle turned off 1. The system skips the shuffle and continues to the next step 2. Refer to UC_006 AC2: Incorrect ballots file for the selected algorithm 1. Tell the user to input the correct ballot file 2. Go to UC_001 AC3: Missing input such as no ballot or candidate to be read 1. Display an error message to the user that the file is missing ballot or candidate to be read 2. Go to back UC_001
Exceptions	EX1: System Fail 7. The system display an error message to the user to try again 8. rerun the system UC_000

4.5 Process Plurality Voting

Name	Process Plurality Voting
ID	UC_004
Description	The Plurality voting system is where each voter selects one candidate, and the candidate(s) with the most votes win. In the case of a tie, a winner is chosen randomly.
Actors	Election official, tester
Organization Benefits	Make the process of determining election results easier Select winners at random in the case of a tie to ensure fairness
Frequency of Use	Once every time the plurality voting format is used
Triggers	The plurality voting system is selected and provides the necessary inputs (see UC_001)
Preconditions	A valid ballot file exists in the CSV file and the number of seats is provided
Postconditions	Display the election result, including the type of election, number of ballots, number of seats, number of candidates, as well as the winners and losers along with their percentage of votes received.
Main Course	1. The system read the candidates' names and ballots (UC_002) 3. The System counts the votes for each candidate 4. The system identifies candidates with the most votes and declares the winner(s) 5. If a tie, the system will randomly select the winner to refer to UC_007 6. The system displays the result as described in the postconditions
Alternate Courses	AC1: Missing input such as no ballot or candidate to be read 1. Display an error message to the user that the file is missing a ballot or candidate to be read 2. Go to back UC_001 AC2: Incorrect ballots file for the selected algorithm 1. Tell the user to input the correct ballot file 2. Go to back UC_001
Exceptions	EX1: System fails to read the file (e.g. file is too large) 1. The system notifies the user that an error occurred 2. Go to UC_001

4.6 Create Audit File

Name	Create Audit File
ID	UC_005
Description	The audit file is created and saved after the STV voting is run. The file includes all the information, including algorithm type, number of ballots, number of seats, winners & losers list, ballot distribution history
Actors	Tester, Election Official
Organization Benefits	Allow users to save election information for later use or for verification purposes
Frequency of Use	Every time an election is run using STV
Triggers	STV is finished processing
Preconditions	The user has entered a file name for the audit file in UC_001
Postconditions	The audit file is saved The audit file contains all the information
Main Course	1. Prompt the user to that the election process is finished 2. The system gathers all the information after the STV is finished processing, including election type, number of ballots, number of seats, winners & losers list, ballot distribution history(STV voting) 4. The system saves the information in a text file 5. Prompt the user that the file has been saved
Alternate Courses	AC1: File already exists 1. Prompt the user that the filename already exists 2. Prompt the user to enter a new name
Exceptions	EX1: System Fail 9. The system displays an error message to the user to try again 10. rerun the system UC_000

4.7 Turn Off Shuffle

Name	Turn off Shuffle
ID	UC_006
Description	In STV, the shuffle feature can be turned off for testing and debugging

	purposes.
Actors	Tester, election officials
Organization Benefits	Allows the user to check the consistency of the result and verify the correctness of the system.
Frequency of Use	Very time when need to verify results or testing/debugging for STV result
Triggers	When the user turns the shuffle feature off
Preconditions	The election is in STV format
Postconditions	The election run without shuffling the ballots The STV voting system processes the ballots in their original order
Main Course	1. The system detected that the shuffle has been turned off 2. The system processes the STV voting (UC_003) without shuffling the ballots
Alternate Courses	AC1: Shuffling is activated 1. The user does not turn off the shuffle feature 2. The system continuously processes the STV voting
Exceptions	N/A

4.8 Handling Tie

Name	Handling Tie
ID	UC_007
Description	In the plurality voting, if two or more candidates receive the same number of votes, the system randomly a winner(s)
Actors	Tester, election officials
Organization Benefits	Making sure it's fair when dealing with tied results
Frequency of Use	Every time there is a tie in the plurality voting
Triggers	A tie occurs after the plurality vote count
Preconditions	The plurality voting system has been selected

	The file has been read successfully (see UC_002) The number of votes per candidate has been calculated
Postconditions	The system resolves the tie between candidates The final winner is determined after resolving the tie The election results are displayed
Main Course	1. The system detects a tie in the voting results 2. The system randomly selects one of the tied candidates as the winner(s) 3. The system continues with UC_004 step 6
Alternate Courses	AC1: No tie occur 1. The system detects no ties in the election results 2. The system declares the candidate with the highest votes as the winner (see UC__003) AC2: Multiple seats with tie 1. Ties occur for multiple seats 2. The system handles each tie independently and randomly selects the winner 3. The system makes sure no candidate is elected more than once
Exceptions	N/A

5. Other Nonfunctional Requirements

5.1 Performance Requirements

The system must process up to 100,000 ballots in under 5 minutes.

5.2 Safety Requirements

The system does not require any safety requirements.

5.3 Security Requirements

There are no security issues to be handled. All security will be handled at the voting precinct.

5.4 Software Quality Attributes

- **Availability**: This system works offline and should be able to process election results whenever an election official needs to process election ballots.

- **Correctness**: The system should process election ballots fairly according to the inputs supplied by the election official.
- **Flexibility:** The system allows the user to select the voting method that they desire

5.5 Business Rules

Only testers can toggle shuffle to assist in unit testing.

6. Other Requirements

There are no other requirements for the voting system.

Appendix A: Glossary

Droop Quota: $Droop\ Quota = \lfloor \frac{numberOfBallots}{numberOfSeats + 1} \rfloor + 1$
Plurality: A voting method where the candidate with the most votes wins.
Single Transferable Voting (STV): A voting system where voters rank candidates in order of preference. To win, the candidate must reach the Droop Quota. This style of voting ensures proportional representation and minimizes wasted votes.

Appendix B: Analysis Models

TBD

Appendix C: To Be Determined List

1. 2.6
2. Appendix B